

Vehicle Re-Identification for Comprehensive Multi-Camera Tracking in Vietnamese Urban Infrastructure

Mã Kim Phát, Đặng Chí Nguyên, and Đường Thị Mộng Thúy

Faculty of Information Science and Engineering, University of Information
Technology, Viet Nam National University, Ho Chi Minh City, Viet Nam
{22521071,22520963,22521454}@gm.uit.edu.vn

Abstract. Vehicle Re-Identification (Re-ID) plays a crucial role in intelligent transportation systems by enabling consistent tracking of vehicles across non-overlapping camera views. This study proposes a comprehensive approach to modify multi-camera vehicle tracking for Vietnamese urban infrastructure, which presents unique challenges such as dense traffic and varied vehicle types. By integrating object detection using YOLOv5 and YOLOv8 with Deep SORT for intra-camera tracking and combining a state-of-the-art Re-ID model (e.g., OSNet) and our proposed method for inter-camera association, our system ensures robust identity preservation throughout the video streams. The system is built on a scalable Apache Spark Streaming pipeline, allowing real-time processing of video frames from multiple cameras via Kafka. Our experimental evaluations on a custom dataset representing Vietnamese streets demonstrate promising performance in terms of accuracy and processing speed.

Keywords: Multi-camera Tracking · Vehicle Identification · Deep-SORT

1 Introduction

During the process of urbanization and the development of intelligent transportation systems, vehicle monitoring and tracking have become increasingly important. Among various approaches, Vehicle Re-Identification (Re-ID) is an effective solution that enables identifying and tracking vehicles as they move across multiple camera views.

However, most existing Re-ID models are developed and tested in ideal conditions, which are not truly suitable for real-world scenarios in Vietnam—where traffic is dense, vehicle types are diverse, and image quality is often affected by noise.

In response to this reality, this project builds a multi-camera vehicle tracking system by integrating modern deep learning models such as YOLOv5, YOLOv8, Deep SORT, and OSNet. The system is deployed on a real-time data processing platform using Apache Spark and Kafka to meet operational requirements in actual Vietnamese traffic conditions.

2 Related Works

Traditional multiple object tracking (MOT) methods often rely on a tracking-by-detection paradigm, where the quality of tracking is strongly influenced by the performance of the object detector. A notable example is the framework proposed by Bewley et al. [1], which introduced a simple yet effective online tracking system emphasizing frame-to-frame association using classical methods such as the Kalman Filter and Hungarian algorithm. Their study demonstrated that with modern, high-quality object detectors, state-of-the-art tracking performance can be achieved without relying on complex models.

To address the limitations of frame-based tracking in handling occlusions, Wojke et al. [2] proposed Deep SORT, an extension of the Simple Online and Realtime Tracking (SORT) framework. While SORT focuses on fast and simple tracking using Kalman filtering and the Hungarian algorithm, Deep SORT incorporates deep appearance features to significantly improve identity preservation. By learning a deep appearance embedding from large-scale person re-identification datasets in an offline stage, Deep SORT can associate detections across longer-term occlusions using nearest-neighbor matching in the learned feature space. This enhancement results in a 45% reduction in identity switches compared to the original SORT, while maintaining real-time performance.

Our work builds upon Deep SORT by adapting it for the vehicle re-identification domain in a multi-camera urban environment. Unlike pedestrian tracking, vehicle tracking in dense Vietnamese traffic requires handling occlusion scenarios and intra-class similarity (e.g., multiple vehicles of the same model or people driving vehicles with similar appearance). We integrate Deep SORT with a modern Re-ID model (e.g., OSNet) trained on vehicle datasets to enable robust cross-camera identity association, addressing the challenges specific to vehicle re-identification across non-overlapping views.

3 Methodology

The Simple Online and Realtime Tracking (SORT) algorithm [1] is a lightweight tracking-by-detection approach that uses a Kalman Filter for motion prediction and the Hungarian algorithm for data association between frames. It relies purely on bounding box information, making it fast and efficient but limited when dealing with occlusions or appearance ambiguities.

Deep SORT [2] extends SORT by incorporating deep appearance features learned from a pre-trained re-identification model. These features allow the tracker to distinguish between visually similar objects and maintain consistent identities across short occlusions. During inference, Deep SORT matches detections to existing tracks by combining motion (from Kalman Filter) and appearance similarity (using cosine distance in embedding space). Moreover, the matching cascade algorithm plays a crucial role that helps Deep SORT improve the performance of SORT.

In our work, Deep SORT serves as the core framework for intra-camera vehicle tracking. We further enhance identity consistency across cameras by integrating a dedicated vehicle Re-ID module and a customized matching strategy designed for urban traffic scenarios. Our pipeline can be seen in Figure 1

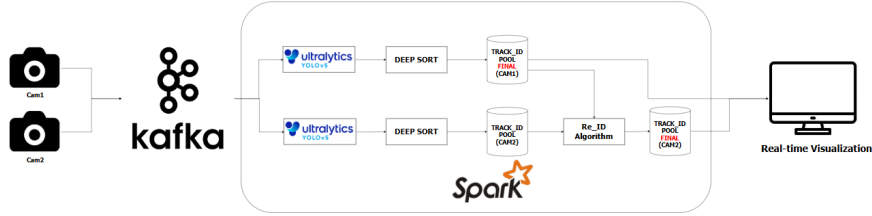


Fig. 1. Our proposed system is a real-time multi-camera vehicle tracking pipeline that combines object detection, tracking, and re-identification, implemented in a distributed streaming architecture using Apache Spark and Kafka.

3.1 ReID Query Algorithm - Our Algorithm

Currently, most object re-identification (Re-ID) systems rely on deep learning models to extract appearance-based features for identity matching. However, training such models from scratch typically requires a large amount of labeled data and substantial computational resources, which can pose challenges for real-world deployments.

In this study, instead of training a deep model from scratch, we adopt a pre-existing architecture and fine-tune it on a vehicle dataset collected from real traffic conditions in Vietnam. This fine-tuning process allows the model to better adapt to local challenges such as diverse vehicle types, complex urban traffic scenes, and non-overlapping camera views. The extracted features are then used in a lightweight identity-matching strategy, referred to as the Re-ID Query Matching Algorithm, which is inspired by the matching cascade algorithm in Zhao et al [3]. Our matching logic combines cosine similarity and a short-term voting mechanism. This approach enables accurate cross-camera identity association without requiring a heavy Re-ID model during inference.

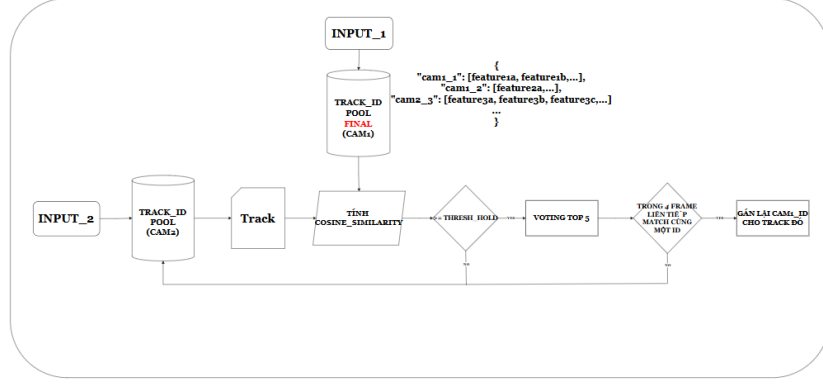


Fig. 2. Our proposed system is a real-time multi-camera vehicle tracking pipeline that combines object detection, tracking, and re-identification, implemented in a distributed streaming architecture using Apache Spark and Kafka.

Algorithm 1 Re-ID Query Matching Algorithm

Input: Feature vector f_q from new track in Cam_2 ;
 Feature pools $\mathcal{P}_{cam1} = \{id_i : [f_{i1}, f_{i2}, \dots]\}$ from Cam_1 ;
 Similarity threshold τ ; Voting window size W ; Majority count P

Output: Assigned Global ID or retain local ID

- 1: Compute cosine similarities $s_i = \cos(f_q, f_i)$ for all $f_i \in \mathcal{P}_{cam1}$
 - 2: Select top-5 most similar IDs with $s_i > \tau$
 - 3: Store top-1 match result into a sliding buffer of size W
 - 4: **if** any ID appears at least P times in the buffer **then**
 - 5: Assign that ID as the global ID for f_q
 - 6: **else**
 - 7: Keep original local ID
-

3.2 Implement in Streaming Data

To deploy our streaming data system, we used two popular platforms: Apache Kafka and Apache Spark. Their combination allowed us to build a robust architecture capable of processing large volumes of real-time video data from multiple cameras. Kafka acts as a tool for collecting and managing data from various camera sources, ensuring durability and fault tolerance. Meanwhile, Spark Streaming is used for distributed and highly scalable processing and analysis of this data.

With this architecture, the entire multi-camera vehicle tracking process, from initial object detection and intra-camera tracking with Deep SORT, to inter-camera identity synchronization through our Re-ID mechanism, is carried out efficiently. This enables our system to continuously process information from complex traffic environments, providing continuous and accurate vehicle tracking of vehicles on a large scale.

We configured Spark to continuously read data streams from Kafka topics. Each Kafka record contains information about individual video frames from surveillance cameras. Frame data is Base64 encoded and transmitted as JSON strings, along with corresponding frame IDs and camera IDs, ensuring flexibility and scalability.

At Spark’s processing layer, a User-Defined Function (UDF) with the Pandas UDF API integrates the YOLOv8 object detection model. This UDF is designed to load the YOLO model only once per Spark executor. The detection results, including bounding box coordinates, confidence scores, class IDs, and class names, are packaged into JSON format.

Next, another critical UDF, is applied to perform Deep SORT tracking and inter-camera Re-ID.

Finally, the output data from the tracking and Re-ID processing, including tracked bounding boxes and vehicle IDs (both local and global), along with the original frame data, is normalized into JSON format and written back to a new Kafka topic (visualization). This creates a rich and fully processed data stream, ready for visualization tasks, and demonstrates the system’s ability to perform real-time, multi-camera vehicle tracking.

4 Experiments

4.1 Dataset

To support the vehicle re-identification task in real-world traffic scenarios, we constructed a custom dataset recorded at two locations in Ho Chi Minh City: An Phu Station and Thao Dien Station. Video data was collected using two smartphones with identical camera specifications, recording simultaneously at both sites to simulate non-overlapping camera views.



Fig. 3. The two locations used for data collection: An Phu Station and Thao Dien Station in Ho Chi Minh City, Vietnam.

The dataset consists of two video clips, recorded at two different locations (one video per location), each lasting approximately 5–6 minutes. The dataset focuses on two main types of vehicles: cars and motorbikes.

Annotations were manually labeled in YOLO format by a team of three annotators, including:

- 1 gold annotator (responsible for high-quality reference annotations)
- 2 regular annotators

After annotation, the dataset was split into three separate subsets for model training and evaluation:

- Training set: 70%
- Test set: 20%
- Validation set: 10%

The data was recorded under clear daylight conditions, with stable lighting and no rain, to ensure consistency and clarity in appearance across both camera views. Annotation followed strict rules: objects with over 60% occlusion were ignored, and only vehicles in the bottom half of the frame were labeled to simulate realistic detection zones.



Fig. 4. The red box illustrates a region that should not be annotated due to occlusion.

Annotation Agreement Metrics

To evaluate the quality and consistency of the manual annotations, we measured the agreement among annotators using a randomly selected subset of labeled data. This process involved comparing annotations made by two regular annotators with a reference (gold) annotator. The metrics were computed based on bounding box overlaps and identity matching, using standard evaluation criteria in object detection and tracking.

- **Average F1-Score:** 90.61%
- **Average Intersection over Union (IoU):** 84.92%
- **Mean Average Precision (mAP) @ [IoU = 0.50:0.95]:** 63.4%

These results confirm that the dataset provides reliable supervision signals for training and evaluating vehicle Re-ID models in real-world conditions.

4.2 Metrics

To evaluate the performance of the YOLO models, we adopted benchmark metrics based on the evaluation described in Jegham, Nidhal, et al. [5].

- **Precision** is the proportion of correctly predicted positive instances among all predicted positives.

$$\text{Precision} = \frac{TP}{TP + FP}$$

- **Recall** is the proportion of correctly predicted positive instances among all actual positives.

$$\text{Recall} = \frac{TP}{TP + FN}$$

- **IoU (Intersection over Union)** is a metric that measures the overlap between the predicted bounding box and the ground truth bounding box.

$$\text{IoU} = \frac{\text{Area of Overlap}}{\text{Area of Union}}$$

- **mAP@50** is the mean of AP across all classes using an IoU threshold of 0.50.

$$\text{mAP}_{@50} = \frac{1}{N} \sum_{i=1}^N \text{AP}_i \quad (\text{with IoU} \geq 0.50)$$

where *AP* is the area under the precision-recall curve for a single class.

- **mAP@50:95** is the average AP over 10 IoU thresholds ranging from 0.50 to 0.95 (COCO-style).

$$\text{mAP}_{@[.50:.95]} = \frac{1}{10} \sum_{k=0}^9 \text{mAP}_{@0.5+0.05k}$$

To evaluate the performance of our deep learning models, we adopt standard metrics commonly used in the evaluation of state-of-the-art (SOTA) approaches. Specifically, we report mean Average Precision (mAP), as well as Cumulative Matching Characteristic (CMC) metrics at Rank-1, Rank-5, Rank-10, and Rank-20. **mAP** measures the overall retrieval precision considering both ranking and matching quality, while **Rank-k** metrics represent the percentage of queries for which the correct identity is found among the top-k retrieved results.

To evaluate the performance of re-identification in a multi-camera deployment setting, we utilize the Confusion Matrix, along with the metrics Recall and Precision. The components of the Confusion Matrix are defined as follows:

- **True Positives (TP)**: The model successfully matches an object that actually disappeared from camera 1 and reappeared in camera 2.
- **False Negatives (FN)**: The model fails to match an object that actually disappeared from camera 1 and reappeared in camera 2.
- **False Positives (FP)**: The model returns a match even though the object did not actually disappear from camera 1 and reappear in camera 2.
- **True Negatives (TN)**: The model returns no match, and in reality, the object did not disappear from camera 1 and reappear in camera 2.

4.3 Experimental Result

We trained two object detection models, YOLOv5 and YOLOv8, using a labeled vehicle dataset as described in Section 4.1. The training process was conducted on Google Colab with a NVIDIA Tesla T4 GPU (16 GB VRAM).

Both models were trained for 50 epochs using the AdamW optimizer with a learning rate of 0.001667 and momentum of 0.9. By keeping the dataset, hardware, and training configurations consistent, we ensured a fair comparison of detection performance between the two YOLO architectures.

Next, we fine-tuned our two re-identification backbone models on the VeRi-776 dataset, and subsequently performed transfer learning using our own dataset, as introduced in Section 4.1. The two models were trained for 60 and 12 epochs,

Table 1. Comparison of detection performance between YOLOv5 and YOLOv8

Metrics	YOLOv5	YOLOv8
Precision	0.9034	0.9630
Recall	0.9190	0.8715
mAP@0.5	0.9716	0.9623
mAP@0.5:0.95	0.6205	0.6119

respectively, using the Adam optimizer. The learning rate schedule followed the strategy proposed by Rong et al. [4]. The training objective was based on the Triplet Loss with a margin of 0.3, which encourages the models to learn discriminative embeddings for vehicle re-identification across different camera views.

Table 2. Comparison of vehicle re-identification performance fine-tuning in VeRi-776 between OSNet and MobileNetV2

Model	mAP	Rank-1	Rank-5	Rank-10	Rank-20
OSNet (osnet_x1_0)	60.2%	88.2%	91.4%	93.9%	96.2%
MobileNetV2 (mobilenetv2_x1_0)	62.2%	88.4%	91.8%	94.6%	97.2%

To evaluate the performance of our pipeline, we conducted experiments on a 20-second video segment. The results showed that the system achieved a **Recall of 83.34%** and a **Precision of 55.56%**, with the corresponding Confusion Matrix illustrated in Figure 5.

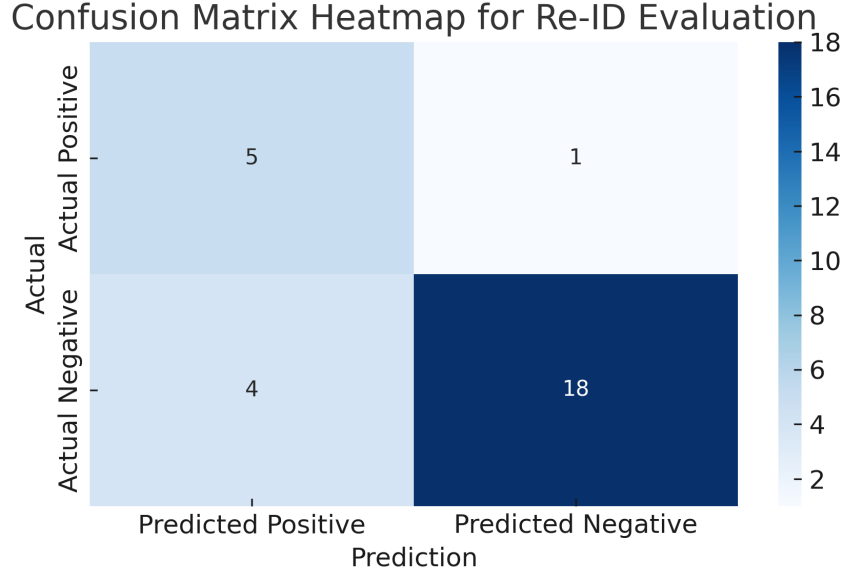


Fig. 5. Confusion Matrix for Re-ID Evaluation

Finally, we integrated two object detection models, YOLOv5 and YOLOv8, with two deep learning-based Re-ID models, OSNet and MobileNetV2, into our streaming data system to evaluate the performance of real-time applications. We deployed the system on a data stream to monitor two 5-minute videos. The table below presents the processing time for each model combination. All experiments were conducted on a server equipped with 16GB RAM, an AMD Ryzen 5 1600 CPU, and a GTX 1060 3GB GPU. The results indicate that the combination of YOLOv8 and MobileNetV2_x1_0 yields the fastest processing time per batch at approximately 7.21 seconds, achieving an average frame rate of around 11 FPS.

Table 3. Comparison of detection performance between YOLOv5 and YOLOv8

Method	Execution Time per Batch	Average FPS
YOLOv5 + Osnet_x1_0	~9.35s	~9fps
YOLOv5 + MobileNetV2_x1_0	~8.1s	~10fps
YOLOv8 + Osnet_x1_0	~7.9s	~10fps
YOLOv8 + MobileNetV2_x1_0	~7.21s	~11fps

5 Future Work

In the future, we will improve our training dataset to better reflect Vietnamese traffic conditions and characteristics, replacing Veri. This will help fine-tune the Re-ID model to be more suitable for urban environments in Vietnam and improve the model's evaluation metrics. At the same time, when deploying Re-ID in a distributed environment (Spark + Kafka), we still encounter issues with ID resets after each Kafka batch and an inability to perform multi-camera Re-ID. We will implement solutions such as connecting to an external database to address these problems.

6 Conclusion

In conclusion, we successfully deployed a cross-camera re-identification system tailored for the Vietnamese context on a big data infrastructure and achieved promising results. However, several limitations remain, including dataset constraints, limited camera angle diversity, and unresolved issues in the system's deployment on streaming data.

References

1. Bewley, Alex, et al. "Simple online and realtime tracking." 2016 IEEE international conference on image processing (ICIP). IEEE, 2016.
<https://arxiv.org/abs/1602.00763>
2. Wojke, Nicolai, Alex Bewley, and Dietrich Paulus. "Simple online and realtime tracking with a deep association metric." 2017 IEEE international conference on image processing (ICIP). IEEE, 2017.
<https://arxiv.org/abs/1703.07402>
3. Zhao, H., Zhai, J. (2022). A Cascade Matching Algorithm for Multi-object Tracking Systems Based on Detection Box Scores. In: Jia, Y., Zhang, W., Fu, Y., Zhao, S. (eds) Proceedings of 2022 Chinese Intelligent Systems Conference. CISC 2022. Lecture Notes in Electrical Engineering, vol 950. Springer, Singapore.
https://doi.org/10.1007/978-981-19-6203-5_85
4. Rong, L., Xu, Y., Zhou, X. et al. A vehicle re-identification framework based on the improved multi-branch feature fusion network. Sci Rep 11, 20210 (2021).
<https://doi.org/10.1038/s41598-021-99646-6>
5. Jegham, Nidhal, et al. "Evaluating the evolution of yolo (you only look once) models: A comprehensive benchmark study of yolo11 and its predecessors." arXiv preprint arXiv:2411.00201 (2024).